Ontology-supported Design Parameter Management for Change Impact Analysis

Jan Novacek*†, Ali Ahari*, Alessandro Cornaglia*, Frederik Haxel*
Alexander Viehl*, Oliver Bringmann*†, Wolfgang Rosenstiel*†
*FZI Forschungszentrum Informatik
Haid-und-Neu-Str. 10-14, 76131 Karlsruhe
†Eberhard Karls Universität Tübingen
Sand 14, 72076 Tübingen

***Abstract*—This paper presents an ontology-supported approach to the management of design parameters in engineering. This approach aims specifically at enabling Change Impact Analysis through Requirements Traceability and acquainted expert knowledge of design parameters. The approach is suitable for both software and hardware designs. The activities and features are mainly obtained by (1) the application of an ontology-based universal system modeling procedure proposal for model integration, (2) the utilization of a knowledge base for capturing expert knowledge and (3) a semantic Mission Profile Aware Design platform. OWL is used to represent information and the underlying data model can improve knowledge transfer among heterogeneous systems which are common in complex engineering projects. At the same time, effort to perform reasoning on such models can be reduced. A demonstration and hands-on description of two illustrative use cases complements the paper. *Index Terms*—Systems Engineering, Knowledge-Based Engineering, Requirements Management, Mission Profiles**

I. Introduction

Requirements imposed on the development of systems frequently change and the consequences of such changes cannot be foreseen. Moreover, frequent changes are the reason why monitoring, traceability and Change Management mechanisms are needed. Requirements Traceability is especially important for the development of safety-critical systems and is required by standards such as the ISO 26262. The Change Impact Analysis (CIA) methodology proposed in this paper uses Trace Links and, in addition, acquainted expert knowledge about the relation between design parameters and requirements. Approaches that consider multiple scopes (as code, architecture and miscellaneous artifacts) are rare. As a consequence, Lehnert concluded that *"[...] more attention should be paid on linking requirements, architectures and code to enable comprehensive impact analysis."* [1].

Typically, requirements are passed along the supply chain while being refined and broken down to separate tier needs, as shown in Fig. 1. Requirements can be linked to one or more *design parameters* which in this context can basically be any characteristic of the artifact or system in question such as the range of a radar module, a system configuration setting or the runtime of a specific function of a software component. The design parameters themselves have interdependencies that are called *operative relationships*. The range of a radar module is for example related to the gain of its amplifier.

Basically, this paper aims at providing support for decision making during development and uses expert knowledge about operative relationships. Employing Trace Links between requirements and design artifacts or representations thereof combined with acquainted expert knowledge about design parameters makes CIA easier and contributes to making the right decision with adequate effort. In addition, formalized knowledge of design parameters and ontological representations of systems are essential for reasoning on system models, e.g. validation, transformation or exchange. In addition, the approach considers Mission Profiles (MPs) in the design process thus enabling Mission Profile Aware Design (MPAD).

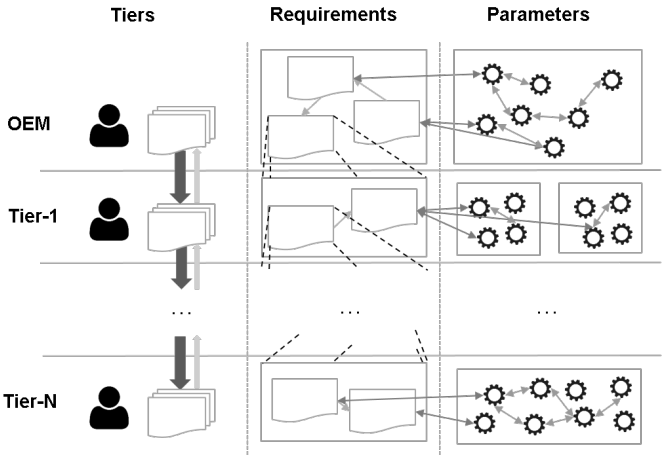

Fig. 1: Supply chain with interdependencies between requirements and design parameters

Operative relationships are mostly unknown. Therefore, consequences from requirement changes might not be taken into consideration and could cause problems due to undesired *side effects* and/or *ripple effects*. In addition, this is why CIA is complex and requires significant effort. As the everyday use of model-based approaches in the automotive and avionic industries is still limited, heterogeneous models and tools are hardly integrated which is identified as a key challenge in engineering settings by Broy et al. in [2]. Borg et al. observe in their comprehensive case study that manual work dominates CIA in industry [3]. This might be a reason for underestimated or unnoticed system impacts and incomplete or delayed analyses [4].

To deal with these issues, the main purpose of the presented approach is the realization of a system that is able to provide an answer to the question "Which design parameters are affected if a requirement is changed?". Providing an answer to this question involves the integration of heterogeneous data sources. Benefits are primarily:

- Avoidance of errors and improvement of overall product quality
- Reduced costs due to the decreased complexity of a CIA
- Improved understanding of the design and the relation between design parameters and requirements

The proposed *system model* used in the approach is ontology-based and follows the principle of Linked Data that allows interlinking and

This is the accepted manuscript of: J. Novacek, A. Ahari, A. Cornaglia, F. Haxel, A. Viehl, O. Bringmann, and W. Rosenstiel, "Ontology-Supported Design Parameter Management for Change Impact Analysis," in 2018 44th Euromicro Conference on Software Engineering and Advanced Applications (SEAA), Prague, Czech Republic, pp. 9–16, 2018. The version of record is available at https://doi.org/10.1109/SEAA.2018.00011.

discovery of new data sources in a global data graph [5]. Relying on an established standard for ontology language (OWL [6]), the system also favors sharing and processing of system models used in development. Creating effective Knowledge Management Systems is a key success factor in engineering process improvement [7].

The main contribution of this paper is the proposed methodology for integrating heterogeneous data sources via the establishment of an ontology-based system model and the use of ontological representations. In addition to data integration applying this methodology enables specification of links between design parameters and requirements as well as traceability information.

This paper is divided into five sections. The Introduction is followed by a section providing relevant background information. The third section describes the solution approach and the fourth section presents two use cases that constitute a hands-on description. The final section consists of a conclusion drawn from the experiences made while carrying out this research.

II. Background and Related Work

This section describes relevant background knowledge. An introduction to MPs and MPAD follows a comparison of the approach with existing work in Knowledge-based and Model-based Systems Engineering. This section also provides some insight to state-of-the-art of ontologies in Requirements Engineering. In addition, there is an outline of the CIA that is used in the presented approach comparing it to related work.

A. Mission Profiles

MPs are used to capture and specify environmental conditions to which components are exposed throughout their life cycle [8]. They contain various relevant stress factors and are therefore composed of different sources such as load usage and environmental profiles, requirements and scenarios for stress tests. MPs play an important role at the design and development of automotive systems, especially with regard to the propagation of requirements along the development process chain [9, 10]. There is no common standard for MPs yet. A format draft to specify MPs has been developed in the RESCAR2.0 project[1] and is further described in [9]. The autoSWIFT project[2] aims at improving and standardizing the format.

B. Mission Profile Aware Design

The general concept of MPAD was introduced by Jerke and Kahng in [11]. According to their paper, MP consideration is still mainly a manual task. Thus, providing tool support is essential for making adoption easier and enabling automation. The presented approach realizes this using the Mission Profile Framework and the semantic MPAD platform as described in Section III-B. The Reliability Knowledge Framework by NXP contains a MP library and was created to connect users, reliability knowledge, data, tools and methods [12]. Today, apart from the mentioned frameworks, there are no other software bundles which provide means to support MPAD so far. The reason might be the fact that a standard for MPs does not yet exist, see Section II-A.

Recently, Hirler et al. presented a theoretical model for reducing stressors in MPs to two single parameters i.e. *effective stress level* and *effective stress time*. Experiments showed that the reliability data obtained fit theoretical predictions within statistical variations [13].

[1]https://www.edacentrum.de/rescar/

[2]https://www.edacentrum.de/autoswift/

C. Knowledge- and Model-based Systems Engineering

Knowledge-based systems support the explicit representation of knowledge in a domain and their exploitation through providing appropriate reasoning mechanisms [14].

Sarder et al. explored the development of Systems Engineering ontologies in [15]. An ontology for Systems Engineering has been proposed by van Ruijven in [16]. Based on the ISO 15926 standard, processes defined in the ISO 15288 standard are modeled. The approach does not directly support MPAD. Nevertheless, the ontology could be used for explicit description of the process presented in this paper using defined semantics.

In the Space Systems domain the application of ontologies in Model-Based Systems Engineering (MBSE) has been explored by Hennig et al. [17, 18]. This approach uses ontologies to describe the system model and the conceptual data model and focuses on the integration of data models of various disciplines. It does not consider MPs and does not implement a CIA approach. Ernadote also presented an approach to support MBSE combining standard meta models with dedicated ontologies to make understanding and creation of models easier [19]. This approach hides the complexity of basic meta models and presents users ontology-based common vocabularies instead. These vocabularies are based on category-theory to support reading and creation of model data. The approach does not implement a CIA, does not provide any means for design parameter management and does not consider MPs.

D. Ontologies in Requirements Engineering

Ontologies in Requirements Engineering (RE) have been used to describe requirements specification documents or for the formal representation of requirements and application domain knowledge.They have also been used to check consistency and completeness [20, 21].

Dermeval et al. pointed out that most studies about the application of ontologies in RE focus on functional requirements [22]. Our approach can handle non-functional requirements, too. The literature review also revealed that only half of the studies followed W3C[3] recommendations on ontology-related languages.

Firesmith pointed out in [23] that the definition of requirements meta data contributes to completeness. Although excluded from the scope of this paper, the use of ontologies in our approach allows specification of machine readable meta data using defined semantics thus enabling reasoning on such data. As Siegemund et al. stated in [21], relationships among requirements are inadequately captured. Ontologies provide the means for formal representation of relations between concepts. As a result, they are suited to define and specify requirement relationships in Requirements Elicitation. Our approach provides the means to lifting requirements descriptions to formal and semantically enriched representations and thereby has the potential to also support various RE activities. Moreover, the work of Siegemund et al. could be used to improve our approach with regard to the requirements representation.

There are various approaches to model requirements. We found that most approaches use some kind of requirement model, although they do apparently not agree on a common type of model except for using ReqIF[4] as a container format, for instance. Nevertheless, ReqIF can transport highly formalized as well as non-formalized requirements. This approach expects requirements to be represented in the most abstract form - natural language. These abstract descriptions are complicated to process and must be lifted to ontological representations

[3]https://www.w3.org/

[4]http://www.omg.org/spec/ReqIF/

first. We do not focus on the lifting itself but on establishing the foundation for semantic enrichment as for example done by Ferdinand et al. in [24].

E. Change Impact Analysis

According to Li et al. CIA techniques can be divided into four perspectives: traditional dependency analysis, software repositories mining, coupling measurement and execution information collection [25]. Although the presented CIA technique can be classified as dependency analysis it is not a traditional (code-based) approach as this paper will show.

Mostly and especially in software CIA, approaches focus on source code (65 % according to Lehnert in [1]) or other expressions such as natural language for instance. Recently, Borg et al. presented an approach with a specific focus on CIA of software artifacts that are not source code in [3]. This approach uses textual content of issue reports as basic information source.

In general, the level of abstraction in CIA can be different and thus not only programming languages but also Hardware Description Languages can possibly be considered even in language-dependent approaches. Lately, CIA for hardware designs has been explored for example by Ring et al. in [26]. His work presented a framework for CIA that focuses on Change Management by detecting relationships between specifications using different levels of abstraction. The paper pointed out that CIA approaches for hardware designs need to take into account multiple levels of abstraction. Although this approach is applicable to hardware designs, it does not consider requirements or MPs.

To date, many CIA approaches mostly focus on functional changes to design artifacts. This might be the reason why the system impact remains underestimated and unnoticed in practice and analyses are incomplete [4]. The approach that is described in this paper can handle both functional and non-functional changes and has a strong focus on the analysis of overall system impact.

Please find more information on software CIA approaches in a detailed overview presented by Lehnert in [1].

III. Approach

This section describes the approach and provides an overview of the concept first. Then follows a explanation of how CIA can be realized applying a *system model* that supports Trace Links and how MPs are integrated. The section concludes with a description of how design parameters are linked to requirements.

A. Overview

A core concept of the approach is the link between design parameters and requirements, see Fig. 1. In our approach, this is achieved through lifting the expert knowledge stored in a knowledge base and the requirements found in requirements specification documents to ontologies, see Fig. 2. Once the ontological representations are ready, links are inserted semi-automatically, as described in Section III-D. As soon as the system model as described in Section III-C, has been established and links to requirements have been inserted, standard graph analysis techniques can be used to determine parameters affected by a change. This basically means collecting all nodes on a path starting from a root node to all leaf nodes.

B. Framework design

This section describes the framework that was designed to provide the means to achieve MPAD tasks. The *Semantic Mission Profile Aware Design Platform* served to accomplish the construction of the presented system but can and will also be used in other settings in the future.

Performing CIA involves connecting and reasoning on multiple heterogeneous data sources. To accomplish this task, using semantic technologies is beneficial as they encourage consolidation of different data sources and improved integration into the processes. The presented approach therefore uses a platform that has been created especially to support MPAD tasks for semantically enriched MPs. The conclusion of this section will give a brief description of the platform and outline the benefits from using it for the presented approach.

1) Architecture and implementation: To accomplish MPAD tasks, a comprehensive framework has been created - the *Mission Profile Framework* (MPF). It allows developers to create and manage MP documents according to the MP format draft mentioned in Section II-A. The platform is implemented as a server application written in the Scala programming language [27] and is meant to be used in conjunction with the MPF but can also be used as a stand-alone application. The platform realizes special MPAD tasks in the form of applications and extends resp. uses the tool and code base that the platform provides.

2) Core functionality: The extendable semantic MPAD platform is designed to process interlink and integrate MPs and requirements which just the MPF alone cannot do. Input documents are therefore lifted to ontological representations by mapping engines. Resulting ontologies contain and allow for the definition of relationships between objects and thus favor semantic processing.

Apart from lifting documents to ontological representations, the platform provides means to integrate Change Management for requirements into workflows. This can be done using a tool that operates with ontological requirements specifications and is able to recognize changes made to them. Using the tool enables automatic reaction to changes and could also be extended to trigger certain actions based on the type of change, for instance.

3) Interfaces: The main intention in the design of a semantic MPAD platform was tight integration into existing design flows. Therefore, the platform provides a RESTful [28] API allowing users direct access to platform features through a web-based application with a HTML and JavaScript GUI. At the same time, the setup burden is reduced. Custom client applications which can fulfill additional special tasks when accessing the platform can use the same API. With regard to requirements, documents in ReqIF format containing requirements specifications are supported directly. All resulting OWL ontologies are serialized to RDF/XML[5] and can be published to user-defined triple stores via the SPARQL Protocol And RDF Query Language[6] or be accessed directly through the platform server API.

C. System model

The system model serves the purpose of specifying various relevant relationships, see Fig. 3 for an example. This model is a means to specify knowledge about the system for model integration, insert Trace Links and support the CIA, as explained in Section III-C1. It is used to specify the relationships

- between *components* and *functions*
- between *components* and *design parameters*
- between *components* and *Mission Profiles*

and for the implementation of Requirements Traceability

- between *systems* and *requirements*
- between *components* and *requirements*.

[5]https://www.w3.org/TR/rdf-primer/

[6]https://www.w3.org/TR/sparql11-overview/

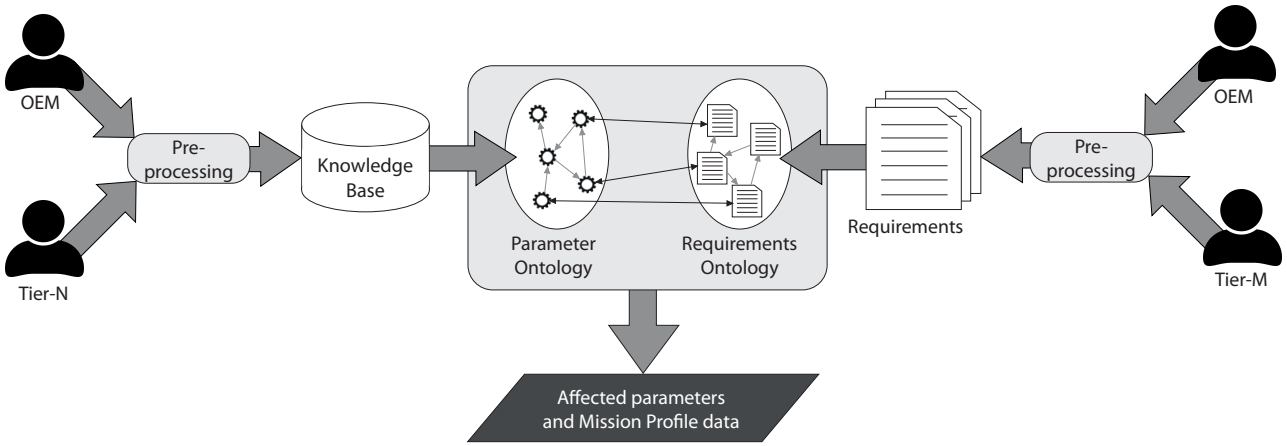

Fig. 2: Overview of the general approach: Expert knowledge about operative relationships between design parameters is lifted to a parameter ontology. Requirements are lifted to a requirements ontology followed by the insertion of links between design parameters and requirements. A subsequent change affecting parameters can be determined and all relevant MP data can be passed to the analyst

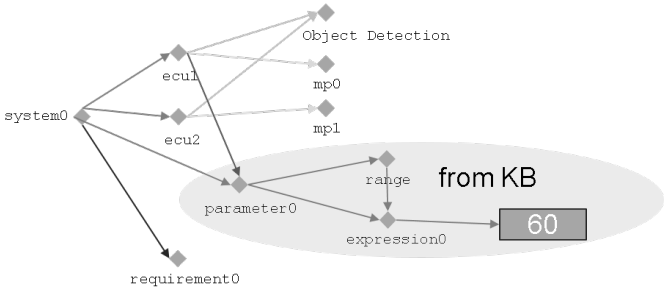

Fig. 3: System model example instance modeling a single instance of a system composed of two Electronic Control Units (ECUs), which realize the *Object Detection* function. The two ECUs are linked to corresponding MPs. One of the ECUs is also linked to a *design parameter* which specifies that the *range* of the component is "60". While the parameter and it's expression stem from the knowledge base, the link between the ECU component and the parameter is user-specified

The system model is expressed as an OWL ontology and is meant to ease model integration by referring to a common uniform model. The primary purposes are:

1) Supporting Change Impact Analysis: The CIA is primarily enabled through specification of relationships between *requirements* and *design parameters* using the system model. These relationships represent Trace Links. We propose a semi-automatic procedure in order to collect these links: candidates for links are suggested to the user who can either accept or decline them. See Section III-D for a detailed description of this methodology.

2) Enabling Requirements Traceability: The system model can be used to enable Requirements Traceability through insertion of Trace Links as mentioned in the previous section.

3) Integrating Mission Profiles: MP data is integrated by inserting links between components and MP document representations in the system model, see also Fig. 3. Ontological MP document representations are used and can be queried for specific data portions.

Data: requirement description $r_i \in \mathcal{R}$, parameter description $p_j \in \mathcal{P}$, $i, j \in \mathbb{N}_0$, $0 \leq i \leq |\mathcal{R}|$, $0 \leq j \leq |\mathcal{P}|$
Result: set of link suggestions $\mathcal{L}$

```
foreach r_i ∈ R do
    foreach p_j ∈ P do
        tokenize r_i and p_j ;
        part-of-speech tagging of all tokens ;
        recognize lemmas of all tokens ;
        foreach token t of tokens of r_i do
            foreach token s of tokens of p_i do
                if t and s are nouns then
                    get lemma l of t and k of s ;
                    get the set S of symmetric relationships
                      between l and k ;
                    if S is not empty then
                        add an entry to the set of link
                          suggestions L with a link between r_i
                          and p_i ;
                    end
                end
            end
        end
    end
end
```

Alg. 1: Link suggestion

D. Linking requirements to design parameters

Design parameters need to be linked to corresponding requirements in oder to use gathered expert knowledge about operative relationships. For this purpose, the proposed system contains a component for the specification of such links and, what is even more important, a component which is able to suggest links to the user providing a semi-automatic way to specify these links.

1) Specification of links: All relevant knowledge (system model, requirements, design parameters) is expressed via ontologies, following W3C recommendations. With regard to the underlying RDF data model, each design parameter or requirement can be seen as a *resource*. A link between these is then represented through a triple

connecting both by a predicate. We identified basically two options for the specification of these links:

(1) indirect specification by generating individuals representing the link:

```
ex:link1 rdf:type ex:Link
ex:link1 ex:hasReq req:requirement1
ex:link1 ex:hasDp dp:designParameter1
```

(2) direct specification by adding a relationship between the requirement and the design parameter instances:

```
req:requirement1 dpr:relatedTo dp:designParamter1
```

The difference between these representations is above all that it is easier to add information to a link in the first form. This is the reason why we decided to represent links using this form.

2) Suggestion of links: To support users in the process of specifying links between requirements and design parameters, the proposed system contains a component which is able to suggest link candidates to the user, see Fig. 7. The user can either accept or decline a link suggestion. Requirements as well as design parameters are expected to be descriptions in natural language. We use Stanford CoreNLP [29] for natural language processing tasks.

The main idea behind our examplary link suggestion is the use of WordNet [30] to identify related synonyms in nouns of natural language requirement respectively design parameter descriptions. This task can be achieved for example with Alg. 1. Although performance could be optimized and it could be extended to compare also the string distances of nouns in lines 8-14 to handle typos in descriptions, for instance, it can be used to find candidates for links in certain situations.

IV. Results

In this section we first describe a framework which was used to capture expert knowledge. Then, we employ our approach to analyze the impact of changes in two use cases. The first use case is based on the power consumption of logic gates and registers model in the TAMTAMS [31] tool. In this use case we focused on the impacted parameters. The second use case is a real-time application in which the real-time requirements are transformed into technology-level requirements. Focus of the second use case was deriving requirements of parameters on a lower level.

A. Capturing expert knowledge

The *Collaborative Technology Evaluation Framework* (CTEF) provides the means to capture expert knowledge about operative relationships and was designed specifically for this task. It represents the knowledge base component in the general approach, see Fig. 2. The centralized system currently contains a server and a client which are loosely coupled and are therefore exchangeable. While the server could for example be hosted by a trusted neutral party, the clients are meant to be used cross-domain by all participants in a supply chain. Fig. 4 shows a screenshot of the CTEF client application.

1) Dependencies and matchings of parameters: Gathered knowledge is stored on the server-side and is represented as a *Dependency Matching Graph* (DMG). Apart from the specification of design parameters which can be set to be either *private* or *public*, CTEF allows for definition of dependencies between design parameters. The definition of relationships between parameters is a fundamental concept of the framework. As this a collaborative framework, these relationships can be either *dependencies* or *matchings*. Matchings are relationships from parameters to parameters of providers. The idea behind is that a component or system might have some top level parameters e.g. the range of a radar system which are themselves depending on other parameters, which need to be supplied by a different provider. A radar system might for example have an amplifier component manufactured by a another provider in the supply chain. As the range of a radar system also depends on the gain of its amplifier, the manufacturer of the amplifier component will also be the *provider* of a corresponding design parameter. A pair of corresponding parameters is a *match* and together with the dependencies forms the DMG, see Fig. 6 for a visualization of a DMG.

2) Transferring knowledge: To actually use the knowledge gathered with CTEF the server component has on the one hand functionality which allows analyzing various aspects of DMG relations and is on the other hand able to export captured information in RDF/XML format. Using the RDF data model constitutes the interface to knowledge bases and systems such as the platform described in Section III-B and is the basis for semantic enrichment and reasoning via the creation of OWL ontologies. In addition, this allows the performance of semantic queries for example with SPARQL against captured knowledge. Introducing this Web-compliance feature allows for interlinking knowledge across deployed CTEF systems on Web-scale to discover and explore most complex operative relationships.

B. Use case: Power consumption

The TAMTAMS module used, is the system level module for analyzing power consumption of logic gates and registers. This module consists of several low level technology parameters, as well as high level parameters such as the frequency or the overall power consumption, see Fig. 5. The dependencies of the parameters form an implicit graph which is explicitly specified using CTEF, see Fig. 6. Please refer to Fig. 7, for an overview of the system components and how they are connected. The grouping of parameters into tiers is just an example and can of course be different in practice. However, this does not affect the approach's outcome. The relationships between design parameters are important and their formation of a hierarchy with design parameters residing on various levels. The set of design parameters and the relationships between them is the knowledge gathered from domain experts via CTEF. In this example, the knowledge about the operative relationships between design parameters is taken from TAMTAMS.

Apart from expert knowledge about operative relationships between design parameters gathered through CTEF, knowledge about requirements is lifted to corresponding ontologies, too. Once the ontologies are ready, the system starts to ingest knowledge about the relationships between requirements and design parameters from user input. The developers establish the system model connecting knowledge and forming the main knowledge corpus. Links between components and MPs are inserted.

Consider the case that the requirement that specifies the system frequency is changed during the development. This is detected as a change to a ReqIF document containing requirements of the system which is being developed. The system resolves the dependencies of the changed requirement to find out which other requirements are affected. As a consequence, the system can detect which design parameters are affected by the change. This is achieved by using the knowledge about operative relationships between design parameters exported as ontology from CTEF and being connected to the system model. Finally, the result is presented to the analyst - affected design parameters: *frequency, ibtb, ion, mobility, vth, sce, qme, pde, dibl, capacitance* and *resistance*.

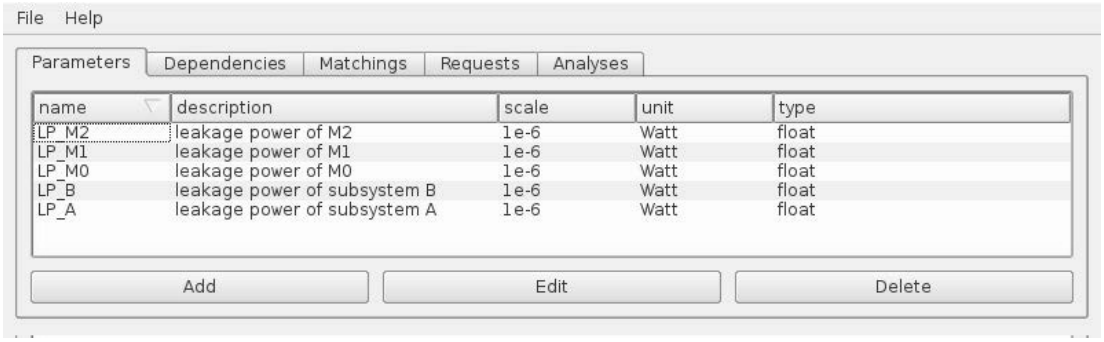

Fig. 4: CTEF client application GUI showing the parameter specification tab which offers the possibility to add, edit and remove parameter definitions. Other tabs include dependency, matching and request management functions as well as utilities for conducting analyses

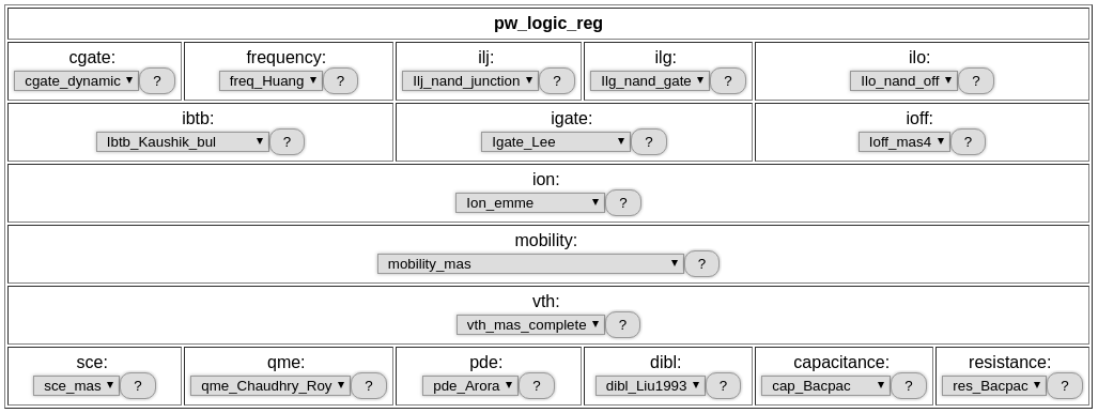

Fig. 5: Web GUI of TAMTAMS for the analysis of the power consumption of logic gates and registers

C. Use case: Real-time application

For further evaluation of the approach we applied it to a traffic sign recognition application which is widely used in automated driving systems. It requires a real-time processing of frames and it runs on an embedded platform. Since there are 24 frames per second there is a real-time requirement which requires that each frame is processed in 42 ms. This requirement can be translated to a requirement of write latency of the memory system.

We trained a model to map the processing time of a single frame to the write latency of the main memory. This was realized by running a vast variety of different benchmarks[7] measuring the performance counters such as the number of cache misses, predictable branches and data read accesses. The model is build up using machine learning to map the performance counters and the write latency to the processing time of a single frame. The results were extracted using the ARM based Lauterbach Trace32 on the Xilinx Zedboard with a Zynq-7000 SoC board equipped with a dual-core Cortex-A9. Based on the processing requirement of 42 ms per frame at 600 Mhz the write latency should not exceed 31 cycles.

Using our proposed system, we can link this requirement to corresponding design parameters. The proposed system also allows specification of dependencies between requirements and taking them into consideration.

[7]http://www.mrtc.mdh.se/projects/wcet/benchmarks.html

V. Conclusion

The paper focused on the identification and specification of links between requirements and design parameters to connect different knowledge sources for the purpose of establishing a common system model containing system knowledge and Trace Links to support CIA. There are possibilities to improve the approach by extending it with methodologies found in the literature, e.g. repository mining for the consideration of the history of changes. The presented approach expects all input to be in the most abstract representation - natural language. If models were used instead, this might ease linking of artifacts as models could be queried for specific attributes used for the decision whether two artifacts relate to each other. This also leads to another opportunity for an improvement: the application of Ontology Mapping or Entity Matching approaches to link requirements to design parameters which is crucial to the practical applicability of our approach.

Further research could be carried out with regard to how reasoning could be used to check for consistency or even to identify links between requirements and design parameters. Important is also the lifting of natural language requirements to formal specifications which is a prerequisite to our approach. It might also be reasonable to explore ways of integration of the presented approach into Requirements Engineering activities such as Requirements Management. Here, the approach could be used to ensure completeness by pointing out missing requirements of design parameters, which was identified as the most difficult task within Requirements Analysis [32].

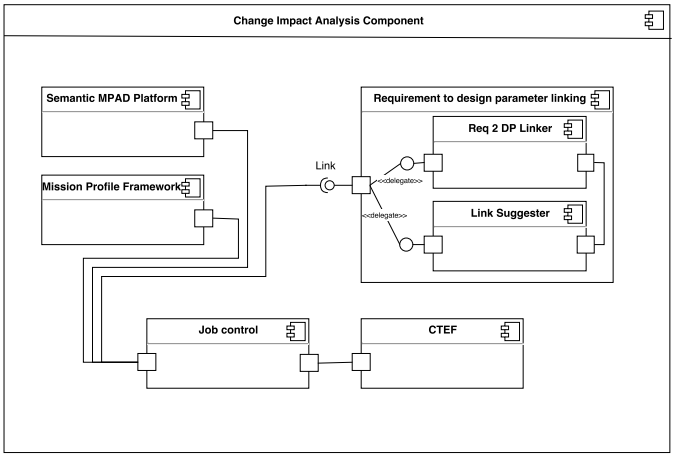

Fig. 7: UML Component diagram of the system realizing the presented approach. All components are connected and controlled by a central control component. Please notice the interface to the *requirement to design parameters linking* component, which is introduced to allow for further experimenting with different approaches in linking

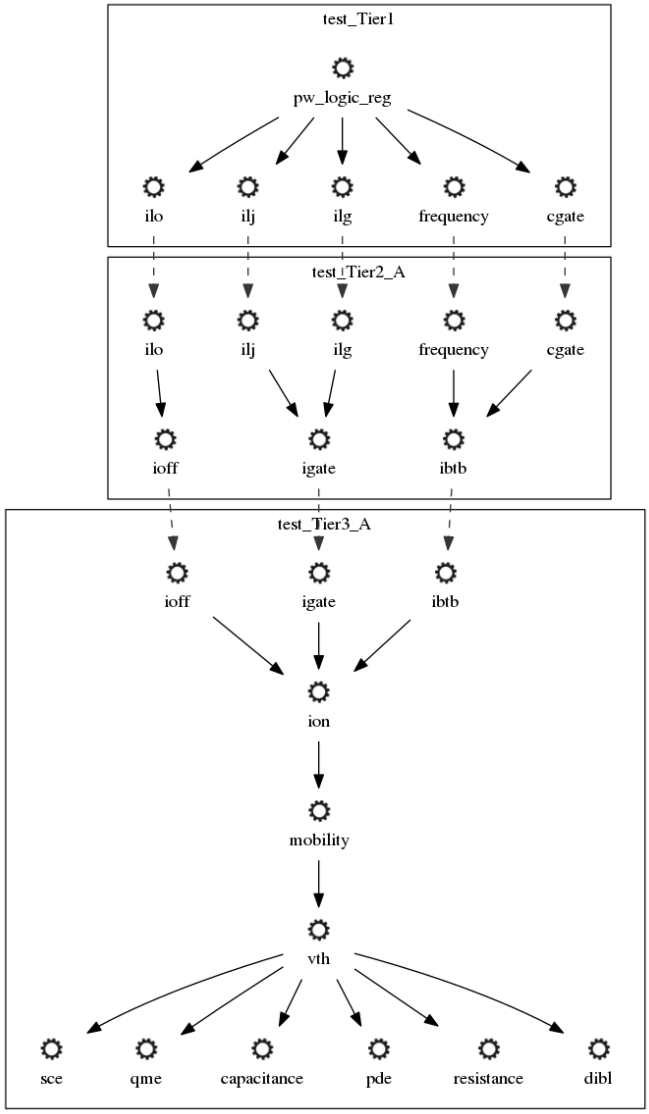

Fig. 6: Translation of the implicit graph structure of the power consumption of logic gates and registers analysis of TAMTAMS to a DMG in CTEF

VI. Acknowledgement

This paper is partially supported by the BMBF projects autoSWIFT (grant number 16ES0358), SAFE4I (grant number 01|S17032C) and the State of Baden-Württemberg, Germany, Ministry of Science, Research and Arts within the cooperative graduate program EAES of the University of Tübingen.

References

[1] S. Lehnert, "A review of software change impact analysis," *Ilmenau University of Technology, Tech. Rep*, 2011.

[2] M. Broy, M. Feilkas, M. Herrmannsdoerfer, S. Merenda, and D. Ratiu, "Seamless model-based development: From isolated tools to integrated model engineering environments," *Proceedings of the IEEE*, vol. 98, no. 4, pp. 526–545, 2010.

[3] M. Borg, K. Wnuk, B. Regnell, and P. Runeson, "Supporting change impact analysis using a recommendation system: An industrial case study in a safety-critical context," *IEEE Transactions on Software Engineering*, vol. 43, no. 7, pp. 675–700, July 2017.

[4] P. Rovegård, L. Angelis, and C. Wohlin, "An empirical study on views of importance of change impact analysis issues," *IEEE Transactions on Software Engineering*, vol. 34, no. 4, pp. 516–530, 2008.

[5] C. Bizer, T. Heath, and T. Berners-Lee, "Linked Data - the story so far," *Semantic Services, Interoperability and Web Applications: Emerging Concepts*, pp. 205–227, 2009.

[6] W3C OWL Working Group, "OWL 2 Web Ontology Language Document Overview (Second Edition) - W3C Recommendation 11 December 2012," 2012.

[7] O. Chourabi, Y. Pollet, and M. B. Ahmed, "Ontology based knowledge modeling for system engineering projects," in *Second International Conference on Research Challenges in Information Science, 2008. RCIS 2008.* IEEE, 2008, pp. 453–458.

[8] C. Byrne and Zentralverband Elektrotechnik und Elektronikindustrie, *Handbook for robustness validation of automotive electrical/electronic modules*. ZVEI, 2008.

[9] T. Nirmaier, A. Burger, G. Harrant, A. Viehl, O. Bringmann, W. Rosenstiel, and G. Pelz, "Mission profile aware robustness assessment of automotive power devices," in *Design, Automation and Test in Europe Conference and Exhibition (DATE), 2014*, 2014, pp. 1–6.

[10] U. Abelein, H. Lochner, D. Hahn, and S. Straube, "Complexity, quality and robustness - the challenges of tomorrow's automotive electronics," in *Design, Automation & Test in Europe Conference & Exhibition (DATE), 2012*. IEEE, 2012, pp. 870–871.

[11] G. Jerke and A. B. Kahng, "Mission profile aware IC design: a case study," in *Proceedings of the conference on Design, Automation & Test in Europe*. European Design and Automation Association, 2014, p. 64.

[12] R. Rongen, "Knowlegde based qualification of semiconductor devices - introduction to the NXP Reliability Knowledge Framework," in *European CEEES Seminar*, 2014.

[13] A. Hirler, J. Biba, A. Alsioufy, T. Lehndorff, T. Sulima, H. Lochner, U. Abelein, and W. Hansch, "Evaluation

of effective stress times and stress levels from mission profiles for semiconductor reliability," *Microelectronics Reliability*, 2017. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S0026271417302111

[14] C. Tasso and E. R. d. A. e Oliveira, *Development of knowledge-based systems for engineering*. Springer, 1998.

[15] M. B. Sarder and S. Ferreira, "Developing systems engineering ontologies," in *System of Systems Engineering, 2007. SoSE'07. IEEE International Conference on*. IEEE, 2007, pp. 1–6.

[16] L. van Ruijven, "Ontology for systems engineering," *Procedia Computer Science*, vol. 16, pp. 383–392, 2013.

[17] C. Hennig, H. Eisenmann, A. Viehl, and O. Bringmann, "On languages for conceptual data modeling in multi-disciplinary space systems engineering," in *Model-Driven Engineering and Software Development (MODELSWARD), 2015 3rd International Conference on*. IEEE, 2015, pp. 384–393.

[18] C. Hennig, A. Viehl, B. Kämpgen, and H. Eisenmann, "Ontology-based design of space systems," in *International Semantic Web Conference*. Springer, 2016, pp. 308–324.

[19] D. Ernadote, "An ontology mindset for system engineering," in *Systems Engineering (ISSE), 2015 IEEE International Symposium on*. IEEE, 2015, pp. 454–460.

[20] V. Castañeda, L. Ballejos, M. L. Caliusco, and M. R. Galli, "The use of ontologies in requirements engineering," *Global journal of researches in engineering*, vol. 10, no. 6, pp. 2–8, 2010.

[21] K. Siegemund, E. J. Thomas, Y. Zhao, J. Pan, and U. Assmann, "Towards ontology-driven requirements engineering," in *Workshop semantic web enabled software engineering at 10th international semantic web conference (ISWC), Bonn*, 2011.

[22] D. Dermeval, J. Vilela, I. I. Bittencourt, J. Castro, S. Isotani, P. Brito, and A. Silva, "Applications of ontologies in requirements engineering: a systematic review of the literature," *Requirements Engineering*, vol. 21, no. 4, pp. 405–437, 2016.

[23] D. Firesmith, "Are your requirements complete?" *Journal of Object Technology*, vol. 4, no. 1, pp. 27–44, 2005.

[24] M. Ferdinand, C. Zirpins, and D. Trastour, "Lifting xml schema to owl," in *International Conference on Web Engineering*. Springer, 2004, pp. 354–358.

[25] B. Li, X. Sun, H. Leung, and S. Zhang, "A survey of code-based change impact analysis techniques," *Software Testing, Verification and Reliability*, vol. 23, no. 8, pp. 613–646, 2013.

[26] M. Ring, J. Stoppe, C. Luth, and R. Drechsler, "Change impact analysis for hardware designs from natural language to system level," in *Specification and Design Languages (FDL), 2016 Forum on*. IEEE, 2016, pp. 1–7.

[27] M. Odersky, P. Altherr, V. Cremet, B. Emir, S. Maneth, S. Micheloud, N. Mihaylov, M. Schinz, E. Stenman, and M. Zenger, "An overview of the scala programming language," EPFL, Lausanne Switzerland, Tech. Rep. IC/2004/64, Tech. Rep., 2004.

[28] R. T. Fielding and R. N. Taylor, *Architectural styles and the design of network-based software architectures*. University of California, Irvine Doctoral dissertation, 2000.

[29] C. D. Manning, M. Surdeanu, J. Bauer, J. R. Finkel, S. Bethard, and D. McClosky, "The stanford CoreNLP natural language processing toolkit," in *ACL (System Demonstrations)*, 2014, pp. 55–60.

[30] G. A. Miller, "WordNet: a lexical database for english," *Communications of the ACM*, vol. 38, no. 11, pp. 39–41, 1995.

[31] M. Vacca, G. Turvani, F. Riente, M. Graziano, D. Demarchi, and G. Piccinini, "TAMTAMS: An open tool to understand nanoelectronics," in *Nanotechnology (IEEE-NANO), 2012 12th IEEE Conference on*. IEEE, 2012, pp. 1–5.

[32] A. M. Davis, *Software requirements: analysis and specification*. Prentice Hall Press, 1990.